LAB: Debugging and Reverse Engineering

4/18/26 4:49 PM

Debugging and Reverse Engineering

1. Immunity Debugger – What It Is & Why Use It

What it is: A free Windows debugger designed for malware analysis. It lets you run a program step-by-step under controlled conditions while watching memory, registers, and instructions in real time.

Why use it:

- Shows exactly what the malware does as it executes (dynamic analysis).
- Reveals hidden behavior that static tools (strings, hashes) miss.
- Great for "living off the land" malware that re-uses Windows APIs.

Difference from IDA Free:

- **IDA Free** = static disassembly (shows assembly code without running the file).
- **Immunity** = dynamic debugging (runs the file and lets you step through assembly line-by-line while it executes).

Best Practice: Use Immunity after static analysis (strings + hash) and before full dynamic tools (Regshot). Always run in a VM with snapshots.

2. The Immunity Debugger Dashboard (Four Main Windows)

When you load any executable (legit or malicious), you see four key sections:

1. **Disassembly Window (upper-left)**
   - Shows the actual assembly instructions the program will execute.
   - Columns:
     - Address (memory location)
     - Opcode (machine code bytes)
     - Assembly instruction (e.g., MOV EAX, 1)
     - Comment (plain-English explanation when available)
   - **What to look for in malware**: API calls (e.g., InternetConnect, RegSetValue, CreateRemoteThread, keybd_event).
2. **Registers Window**
   - Shows current values in CPU registers (EAX, EBX, ECX, EDX, ESP, EBP, etc.).
   - **Why it matters**: Malware often uses registers to store data, pointers, or return values from API calls.
3. **Hex Dump Window**
   - Shows raw memory in hex + ASCII.
   - **Why useful**: Quickly spot strings, URLs, or data the malware is working with.
4. **Stack Window (lower-right)**
   - Shows the program's call stack (function calls and local variables).
   - **Why useful**: Reveals how functions are called and what data is being passed.

Quick Views to Open:

- **View → Log**: Shows loaded DLLs and thread info.
- **View → Memory**: Shows memory regions and permissions (look for RWX = read-write-execute, common for injected code).
- **View → Executable modules**: Lists all DLLs and executables the program loads.

3. Analyzing a Binary File (Example: ieinstal.exe)

Steps (reusable for any file):

1. File → Open → select the executable.
2. Let it load (you'll see the four windows).
3. **View → Log** → see loaded DLLs and threads.
4. **View → Memory** → check for suspicious memory blocks.
5. **View → Executable modules** → list of DLLs the program uses.
6. **Debug → Restart** to run it again cleanly.

What you learn:

- Which Windows APIs/DLLs the program calls (clue to its capabilities).
- Whether it uses "living off the land" (re-using legitimate Windows components).

4. Analyzing a Malware File (Example: test.exe)

Goal: Identify the malware's main functions by watching it run.

Steps:

1. Run Immunity as Administrator.
2. File → Open → select test.exe.
3. Immediately check:
   - **View → Log** → loaded DLLs.
   - **View → Memory** → look for blocks owned by suspicious DLLs (e.g., AcGeneral.dll).
   - **View → Executable modules** → confirm loaded modules.

Key Finding Example:

- AcGeneral.dll with Read/Write/Execute permissions → this DLL can record keyboard and mouse input (keylogger/spyware capability).

What to look for in malware analysis:

- Suspicious DLLs (especially those related to keyboard, network, registry, or process injection).
- Memory regions with RWX permissions (classic sign of code injection).
- API calls visible in disassembly (network, file creation, registry changes).

5. Assembly Basics for Malware Analysis (Simple Explanation)

You don't need to read every line — focus on patterns:

- **Common instructions to watch**:
  - MOV / PUSH / POP → moving data around.
  - CALL → calling a function (especially Windows APIs).
  - JMP / JNZ → jumps/conditional logic.
- **What to look for in the Disassembly window**:
  - Calls to known bad APIs: InternetOpen, RegCreateKey, CreateFile, WriteProcessMemory, keybd_event.
  - Loops or repeated calls → beaconing or keylogging.
  - String references → URLs, file paths, registry keys.

Why assembly matters:

- Malware hides its true purpose in compiled code.
- Assembly shows the exact steps the malware takes (e.g., "open registry key → write value → hook keyboard").

Practical Tip: Scroll to the top of disassembly, look at the first few CALL instructions, and check the comment column for plain-English hints.

6. Last Challenge – Specific Checks

Check for data exfiltration:

- Look in **Memory** or **Log** for network-related DLLs or calls.
- In disassembly, search for InternetConnect, HttpSendRequest, or similar.

Check for changes to the Windows registry:

- In **Memory** view, look for registry-related DLLs (advapi32.dll).
- In disassembly, search for RegCreateKey, RegSetValue, RegOpenKey.

Check if AcGeneral.dll is monitoring inputs:

- Confirmed if you see it loaded in **Memory** with RWX permissions.
- In disassembly, look for keyboard/mouse APIs (e.g., SetWindowsHookEx, keybd_event).

Research loaded DLLs:

- Google each DLL name + "malware" or check VirusTotal.
- Common suspicious ones: anything related to keyboard hooks, remote access, or process injection.

Quick Workflow for Any Malware:

1. Load in Immunity → check Log, Memory, Executable modules.
2. Look for suspicious DLLs and RWX memory.
3. Scan disassembly for dangerous API calls.
4. Step through (F7 = step into, F8 = step over) if needed.

SQL Injection

4/25/26 11:09 PM

What it is

- **SQL Injection (SQLi)** = user input becomes part of SQL query → attacker controls query.
- Root cause: **string concatenation with untrusted input**

Vulnerable pattern

```
SELECT * FROM articles WHERE id = <user_input>
```

Attack example

```
99'; DROP TABLE articles; --
```

Key SQL Characters (used in attacks)

- ' → break out of string
- -- / # / /* */ → comments (ignore rest of query)
- %, _ → wildcards (LIKE)
- () → grouping
- \ → escape

Prevention (what to expect)

- ❌ Escaping → weak
- ✅ **Parameterized queries / prepared statements** (best)
- ✅ ORM (if not using raw SQL)

Types of SQL Injection

1. In-band (most common)

- Immediate feedback (errors/data)
- Used in most demos

2. Blind (inferential)

- No direct output
- Use:
  - response time
  - behavior differences

3. Out-of-band (rare)

- DB interacts externally
  - write files
  - HTTP requests

Detection Workflow (VERY IMPORTANT)

1. **Find inputs**
   - GET, POST, headers
2. **Send payload**
   - ', OR 1=1, UNION
3. **Analyze**
   - errors?
   - different output?
4. **Confirm exploit**
   - login bypass
   - data access
   - side effects

Demo Techniques (What You Need to Know)

1. Login Bypass (Classic)

Payloads:

```
' --
```

→ comments out password check

```
' OR 1=1 --
```

→ always true → logs in as first user (often admin)

```
' OR ''='' --
```

Key Idea:

- Break query logic → force TRUE condition

2. UNION Attack (Extract / Fake Data)

Goal:

- Combine attacker-controlled data with real query

Steps:

1. Find column count:

```
' UNION SELECT 1,2,3 --
```

→ adjust until no error

2. Inject data:

```
' UNION SELECT 'user','pass',1 --
```

Result:

- App trusts DB → logs in as fake user

3. Retrieve Hidden Data

Bypass filters:

```
' OR 1=1 --
```

Remove conditions:

```
' /*
```

→ comments out restrictions like:

```
AND isAvailable = 1
```

Result:

- Hidden/filtered data exposed

4. Extract Passwords (Advanced)

Technique: Boolean-based extraction

```
' AND password > '7' --
```

Idea:

- Compare values → infer characters
- Repeat → reconstruct hash

Faster method:

```
UNION SELECT password AS name ...
```

→ display password hash directly in UI

5. Blind SQL Injection

No errors → use behavior

Time-based:

```
AND benchmark(5000000, sha1('test')) --
```

→ slow response = success

Logic-based:

```
7-4
```

→ evaluated as 3 → proves injection works

6. Out-of-Band SQLi

Example (MySQL):

```
SELECT 'php code' INTO OUTFILE 'c:/path/file.php'
```

Result:

- Writes file to server
- Can execute code via web server

NoSQL Injection (MongoDB)

Concept:

- Still query-based → still injectable

Example:

```
?id[$gt]=3
```

→ becomes:

```
{ id: { $gt: 3 } }
```

Result:

- Returns unintended data (e.g., hidden records)

ORM Injection (IMPORTANT for exams/interviews)

Even ORMs can be vulnerable if using raw queries:

Red flags:

- .raw()
- ExecuteRawQuery
- FromSqlRaw
- createQuery() with string concat
- createSQLQuery()

Rule:

If user input is concatenated → SQLi still possible

Key Attack Patterns (Memorize)

- ' OR 1=1 --
- ' --
- ' UNION SELECT ... --
- ' AND condition --
- /* comment */

What Actually Matters (TL;DR)

- SQLi = **user input → SQL query**
- Look for:
  - login forms
  - search fields
  - IDs in URLs
- Use:
  - ' → test
  - OR 1=1 → bypass
  - UNION → extract data
- Blind:
  - time delays
  - logic tricks
- Always confirm:
  - login bypass
  - extra data
  - side effects

Malicious Activity & Detection

4/15/26 2:13 PM

Core Mindset

Goal: Detect malicious activity at different layers (file, network, logs). **Why**: Malware leaves traces in files, network traffic, and system logs. You combine all three for strong detection.
Three Detection Layers:

- **YARA**: "Does this file look like malware?" (static patterns)
- **Suricata**: "Is this network traffic suspicious?" (behavior in transit)
- **Sigma**: "Did this system behavior happen?" (logs / events)

Key Principle: Focus on **behavior patterns**, not just IOCs. Combine layers for better accuracy.

YARA – File Detection

What it is: Pattern-matching tool for files (and memory). **When to use**: Quick static analysis of suspicious files or binaries. **Why**: Fast way to find malware by matching strings, byte sequences, or code patterns.

Basic Rule Structure

yara

```
rule RuleName {
    meta:
        description = "Detects Azorult stealer"

    strings:
        $x1 = "cmd.exe /c timeout 3 & del" wide      // required
        $x2 = "%TEMP%\\curbuf.dat" wide
        $s1 = "SOFTWARE\\Microsoft\\Windows NT" wide // optional
        $s5 = "Software\\...\\Uninstall\\" base64

    condition:
        uint16(0) == 0x5A4D and          // PE file (MZ header)
        all of ($x*) and
        4 of ($s*)
}
```

Key Concepts & Commands

- **Strings**: What to match on.
  - $x* = required (highly unique)
  - $s* = optional (contextual)
- **Modifiers**:
  - wide → UTF-16 (common in Windows malware)
  - nocase → case insensitive
  - base64 → match encoded version
  - ascii → normal text
- **Condition**:
  - uint16(0) == 0x5A4D → PE file check
  - all of ($x*) → must match all required
  - 4 of ($s*) → flexible matching

Command:
Bash

```
yara rule.yar sample.exe
```

When to use hex patterns instead of strings:

- No readable strings (packed/obfuscated)
- Detect packers (e.g., PECompact2)
- Use IDA Pro to find unique code sequences at entry point

Main Takeaway: YARA = fingerprint files by content. Balance unique + contextual strings.

Suricata – Network Detection

What it is: IDS/IPS for network traffic. **When to use**: Analyze PCAPs or live traffic for C2, downloads, exfiltration. **Why**: Reveals what malware does over the network (even if file is obfuscated).

Basic Rule Structure

suricata

```
alert http $HOME_NET any -> $EXTERNAL_NET any (
    msg:"Amadey C2 Check-in";
    flow:established,to_server;
    http.method; content:"POST";
    http.uri; content:"index.php"; endswith;
    http.header_names; content:!"Referer";
    http.request_body; content:"st="; offset:0; depth:3;
    sid:1000;
)
```

Key Concepts & Commands

- **Action**: alert (generate alert), drop (block)
- **Header**: protocol, IPs, ports, direction (->)
  - $HOME_NET = internal
  - $EXTERNAL_NET = internet
- **Sticky Buffers** (modern way):
  - http.method; content:"POST";
  - http.uri;
  - http.request_body;
  - tls.sni;
- **Modifiers**:
  - nocase
  - startswith / endswith
  - offset / depth (absolute)
  - distance / within (relative)
- **Flow**:
  - flow:established,to_server; → client to server only

TLS Detection (encrypted traffic):

- Use metadata:
  - tls.sni → domain
  - ja3 / ja3s → fingerprints
  - tls.cert_issuer → self-signed certs

Flowbits (correlate rules):

- Rule 1: flowbits:set,c2_active; noalert;
- Rule 2: flowbits:isset,c2_active;

Command:
Bash

```
suricata -S custom.rules -r sample.pcap
```

Main Takeaway: Suricata = detect behavior in network traffic. Use sticky buffers + flow for precision.

Sigma – Log Detection

What it is: Universal rule format for logs (write once, convert anywhere). **When to use**: Detect suspicious system behavior in Windows events / SIEM. **Why**: Vendor-agnostic. Convert to Splunk, Elastic, PowerShell, etc.

Basic Rule Structure

YAML

```
title: Suspicious Office Spawning PowerShell
status: experimental

logsource:
  category: process_creation
  product: windows

detection:
  selection:
    ParentImage|endswith: winword.exe
    Image|endswith: powershell.exe
    CommandLine|contains|all:
      - "-e"
      - "hidden"
      - "-nop"

condition: selection
```

Key Concepts

- **logsource**: Defines data (category, product, service)
- **detection**:
  - selection: your logic (maps or lists)
  - Modifiers: contains, endswith, startswith, all
- **condition**: condition: selection → "alert if selection matches"

Conversion (MUST DO):
Bash

```
sigma plugin install powershell
sigma convert --target powershell rule.yml
```

→ Turns Sigma into real query (PowerShell, Splunk SPL, Elastic KQL)
Common Patterns:

- Process creation: Parent = winword.exe + Child = powershell.exe + encoded args
- Persistence: schtasks.exe /Create /TR from Temp
- Ransomware: file creation ending in .README.txt in Desktop/Downloads

Main Takeaway: Sigma = log behavior detection. Write once → convert to any SIEM.

How the Three Layers Work Together

Attack Chain Example (Malicious Document):

1. **YARA**: Detects malicious Word doc or embedded payload (strings, MZ header).
2. **Suricata**: Detects network activity (PowerShell download, C2 POST, DLL fetch).
3. **Sigma**: Detects system behavior (Word → PowerShell with -e hidden -nop, schtasks, ransom note).

Why combine them:

- YARA catches the file
- Suricata catches the network
- Sigma catches the host impact
- Together = full visibility + fewer false positives

Detection Mindset:

- Start with behavior (what malware does)
- Choose the right layer
- Build precise patterns
- Test & tune (goodware vs malware)

Overall Detection Workflow

1. **Analyze sample** — sandbox (behavior), strings, PCAP, logs.
2. **Identify patterns** — unique strings, commands, traffic, file names.
3. **Choose layer**:
   - File → YARA
   - Network → Suricata
   - Logs → Sigma
4. **Write rule** — use required + contextual indicators.
5. **Test**:
   - YARA: yara rule.yar file
   - Suricata: suricata -S rules -r pcap
   - Sigma: convert → run query
6. **Tune** — reduce false positives, improve specificity.
7. **Document** — IOCs, family, behavior.

Final Takeaways:

- YARA = file fingerprints
- Suricata = network behavior (use sticky buffers + flow)
- Sigma = log behavior (convert before use)
- Best detection = combine all three layers
- Focus on behavior chains, not single IOCs
- Always test rules (goodware + malware)

Assembly for Malware Analysis

4/15/26 2:31 PM

1. CPU Architecture & Number Systems

CPU basics (what malware interacts with):

- **Registers**: Very fast storage inside the CPU. Malware constantly moves data in and out of registers.
  - Common 32-bit: EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP
  - 64-bit version: RAX, RBX, etc.
  - EAX/RAX often holds return values from functions.
  - ESP/RSP = stack pointer (points to top of stack).
  - EBP/RBP = base pointer (helps access local variables and parameters).

Example in assembly (what you see in IDA Pro):

```
mov eax, 5          ; put 5 into EAX register
mov ecx, [ebp+8]    ; read parameter from stack into ECX
```

Memory: RAM where variables, code, and the stack live. Malware reads and writes memory constantly.

Stack: Special area of memory used for function calls, local variables, and saving return addresses. It grows downward (lower addresses).

Number systems:

- **Hex** (base 16): Most common in disassembly. Every byte = two hex digits.
- **Why hex?** Easy to read raw bytes and addresses.

Example:

```
0x4D5A    ; "MZ" - start of every Windows executable (PE header)
```

Malware tip: When you see 0x in IDA, it is hex. You will use hex for addresses, opcodes, and constants.

2. Basic Assembly Instructions

Assembly instructions tell the CPU what to do. In malware analysis you read these in IDA Pro to understand logic.

Most common instructions (memorize these):

- MOV dest, src -> copy value

```
mov eax, 10         ; EAX = 10
mov ebx, eax        ; EBX = EAX
```

- PUSH value -> put value on stack (stack grows down)

```
push 5              ; put 5 on stack
```

- POP dest -> take value from stack

```
pop eax             ; take value from stack into EAX
```

- CALL function -> call a function (pushes return address)

```
call sub_401000     ; call function at address 401000
```

- RET -> return from function

```
ret                 ; pop return address and jump back
```

- ADD / SUB -> math

```
add eax, 5          ; EAX = EAX + 5
sub esp, 0x20       ; allocate 32 bytes on stack for locals
```

- CMP op1, op2 -> compare (sets flags)

```
cmp eax, 5          ; compare EAX with 5
```

- JMP label -> unconditional jump

```
jmp end_loop        ; always jump to end_loop
```

- Conditional jumps (jcc):

```
je  equal_label     ; jump if equal (from CMP)
jne not_equal       ; jump if not equal
jg  greater         ; jump if greater
jl  less            ; jump if less
```

- Malware tip: In IDA Pro, blue arrows = jumps, red arrows = calls. Follow them to understand control flow.

3. Functions & The Stack

Stack basics:

- Used for local variables, function parameters, and return addresses.
- ESP/RSP always points to the top.

Typical function structure (what you see in every malware function):

```
sub_401000:             ; function start
    push ebp            ; save old base pointer
    mov ebp, esp        ; set new base pointer
    sub esp, 0x20       ; allocate space for local variables

    ; function body here

    mov esp, ebp        ; clean up stack
    pop ebp             ; restore old base pointer
    ret                 ; return
```

Calling conventions (how arguments are passed):

- **cdecl** (common in 32-bit malware): Arguments pushed on stack right-to-left. Caller cleans stack (add esp, X).
- **x64** (64-bit): First 4 arguments in registers (RCX, RDX, R8, R9), rest on stack.

Malware tip: Look for push ebp; mov ebp, esp — this marks the start of a function in IDA Pro.

4. Control Flow (Ifs, Loops, Switches)

If statements:

```
cmp eax, 5          ; compare
jne else_part       ; if not equal, jump to else
; if true code here
jmp end_if
else_part:
; else code here
end_if:
```

Loops (for / while):

```
mov ecx, 0          ; i = 0
loop_start:
    cmp ecx, 10
    jge loop_end    ; if i >= 10, exit

    ; loop body

    inc ecx         ; i++
    jmp loop_start
loop_end:
```

Switches: Often use a jump table (array of addresses). Malware uses switches for command handling or anti-analysis.

Malware tip: In IDA Pro, look for CMP followed by conditional jump (jcc). Upward jumps = loops. Downward jumps = if/else.

5. Arrays, Structures & Pointers

Pointers: A value that holds a memory address.

Example:

```
lea eax, [string]       ; load address of string into EAX (pointer)
mov ebx, [eax]          ; read value at that address
```

Arrays: Contiguous memory. Accessed with offset.

Example:

```
mov eax, [commands + ecx*4]   ; read 4th element (ecx=3) of array
```

Structures: Group of variables. Accessed with fixed offsets.

Example (Config struct):

```
mov edx, [ebp-0x10]     ; cfg.port (offset 0x10)
mov ebx, [ebp-0x0C]     ; cfg.url  (offset 0x0C)
```

Malware tip: In IDA Pro, press T on a register to apply a structure. Repeated offsets (+4, +8, +0xC) often mean array or struct.

6. Complete Example: C Code vs Assembly

C Code (covers every concept in the course):

C Copy

```c
#include <stdio.h>

struct Config {
    int port;
    char *url;
};

int checkAuth(int userID) {
    if (userID == 1337) {
        return 1;       // success
    }
    return 0;
}

int main() {
    struct Config cfg = {443, "http://c2.example.com"};
    int i;
    char *commands[3] = {"download", "exec", "exfil"};

    for (i = 0; i < 3; i++) {
        if (checkAuth(1337)) {
            printf("Running %s on port %d\n", commands[i], cfg.port);
        }
    }
    return 0;
}
```

Corresponding Assembly (simplified, as seen in IDA Pro):

asm Copy

```
main:
    push ebp                    ; function prologue
    mov ebp, esp
    sub esp, 0x20               ; allocate stack for locals

    ; struct Config cfg = {443, "http://c2.example.com"}
    mov dword ptr [ebp-0x10], 443   ; cfg.port = 443
    lea eax, [c2_url]               ; load address of string (pointer)
    mov [ebp-0x0C], eax             ; cfg.url = pointer

    ; for loop (i = 0; i < 3; i++)
    mov dword ptr [ebp-4], 0        ; i = 0
for_start:
    cmp dword ptr [ebp-4], 3
    jge for_end                     ; if i >= 3, exit loop

    ; if (checkAuth(1337))
    push 1337
    call checkAuth                  ; call function
    add esp, 4                      ; cdecl: clean stack
    cmp eax, 1
    jne skip                        ; if not equal (false), skip

    ; printf call (simplified)
    push [ebp-0x10]                 ; cfg.port
    mov ecx, [ebp-4]
    lea edx, [commands + ecx*4]     ; array access with offset
    push dword ptr [edx]            ; commands[i]
    call printf
    add esp, 8

skip:
    inc dword ptr [ebp-4]           ; i++
    jmp for_start                   ; loop back

for_end:
    xor eax, eax                    ; return 0
    mov esp, ebp
    pop ebp
    ret

; checkAuth function
checkAuth:
    push ebp
    mov ebp, esp
    mov eax, [ebp+8]                ; userID parameter (from stack)
    cmp eax, 1337
    jne fail
    mov eax, 1                      ; return 1
    jmp done
fail:
    xor eax, eax                    ; return 0
done:
    mov esp, ebp
    pop ebp
    ret
```

Line-by-Line Explanation (how it connects to the course):

- `push ebp; mov ebp, esp; sub esp, 0x20` : Function prologue + stack allocation (functions & stack).
- `mov dword ptr [ebp-0x10], 443` : Writing to structure field using offset (structures).
- `lea eax, [c2_url]` : Loading address into register (pointers).
- `cmp ... jge for_end` : Loop condition (loops).
- `cmp eax, 1; jne skip` : If statement (comparisons & jumps).
- `call checkAuth` : Function call (functions & calling conventions).
- `add esp, 4` : Caller cleaning stack (cdecl convention).
- `mov ecx, [ebp-4]; lea edx, [commands + ecx*4]` : Array access with index (arrays).
- `ret` : Return from function.

Malware analysis tip: Real malware looks similar but with:

- Obfuscated names
- More stack manipulation
- Calls to Windows APIs (CreateProcess, InternetConnect, etc.)
- Heavy use of `CMP + Jcc` for control flow

LAB: Static/Dynmaic Analysis

4/18/26 4:18 PM

Overall Malware Analysis Workflow

1. **Static** → Strings + Hash + VT (quick IOCs, no risk).
2. **Dynamic** → Regshot snapshots (see real changes).
3. **Decomposition** → IDA disassembly (understand code logic and APIs).

Safety First:

- Always use isolated VM with snapshots.
- Never run on production systems.
- Document everything (hash, behavior, changes).

Key Takeaways:

- Static = safe & fast.
- Dynamic = reveals actual impact.
- Disassembly = deepest code-level understanding.
- Combine all three for complete picture.

1. Static Analysis (Without Running the Malware)

What it is: Examine the file as-is to extract strings, hashes, imports, and basic indicators.
Why it matters: Fast, safe, no risk of infection. Good first step to decide if deeper analysis is needed.
Key Tools & Commands:

- **strings** (strings64.exe on Windows):
  cmd
  ```
  strings Formbook.exe > strings.txt
  ```
  - Extracts printable ASCII/Unicode strings.
  - Look for: DLL names, function imports, URLs, file paths, registry keys, embedded executables (e.g., pRue.exe).
- **certutil** (built-in Windows):
  cmd
  ```
  certutil -hashfile sample.exe SHA256
  ```
  - Generates unique file hash (SHA256 recommended).
- **VirusTotal**:
  - Search by hash.
  - Check tabs: **Behavior** (file drops, network activity), **Network Communication** (C2 domains/IPs), **File System Actions** (created/modified files).

What to Look For:

- Suspicious strings (DLLs, .exe names, URLs).
- Hash for threat intelligence.
- Behavioral summary from VT (even before running it).

Interview tip: “Static analysis gives quick IOCs and strings without execution risk.”

2. Dynamic Analysis (Observe Changes While Running)

What it is: Run the malware in a controlled environment and compare system state before/after.
Why it matters: Reveals exact changes (registry, files, processes) that static analysis misses. Shows real behavior.
Key Tool: Regshot (simple before/after snapshot):
Steps:

1. Take **1st Shot** (baseline of clean system).
2. Run malware (as admin if needed).
3. Wait for execution.
4. Take **2nd Shot**.
5. **Compare** → generates report.

What the Report Shows:

- **Keys added** / **Values added**: New registry entries (persistence).
- **Values deleted** / **Values modified**: Changes to existing settings.
- **Total changes**: Overall impact summary.

Reusable Tips:

- Always run in isolated VM with snapshot.
- Focus on registry (Run keys, services) and file drops.
- Compare multiple runs — behavior can vary.

Interview tip: “Dynamic analysis with snapshot tools like Regshot shows precise filesystem and registry changes caused by malware.”

3. Malware Decomposition / Disassembly

What it is: Break the executable into assembly instructions to understand its low-level logic.
Why it matters: Reveals exact code flow, functions, and hidden behavior (especially in packed or obfuscated samples).
Key Tool: IDA Free (Interactive Disassembler):
Basic Workflow:

1. Load the executable (File → New).
2. Let IDA analyze (auto-disassembly).
3. Explore views:
   - **Functions window**: List of functions (start with start or main).
   - **Disassembly view**: Assembly instructions (pin to keep visible).
   - **Hex View**: Raw bytes/hex of the code.
   - **Imports tab**: DLLs and API functions the malware calls (critical for understanding capabilities).
   - **Exports tab**: Functions exported by the sample (less common in malware).

What to Look For:

- Imported APIs (e.g., network, registry, process creation).
- Entry point and main logic flow.
- Strings or constants that were hidden in static strings output.

Reusable Tips:

- Start with Imports tab — tells you what the malware *can* do.
- Look for suspicious calls (CreateProcess, InternetConnect, RegSetValue).
- Use for packed malware: unpack first, then disassemble the unpacked binary.

Interview tip: “Disassembly with IDA shows the exact assembly instructions and imported APIs, giving deep insight into malware capabilities without full execution.”

Initial File Triage

4/15/26 1:57 PM

Overall Triage Workflow(Summary)

Step-by-step order:

1. **Safe handling** — ZIP + infected password, rename extension.
2. **Hash the file** — sha256sum → search VirusTotal/MalwareBazaar.
3. **Run in sandbox** (ANY.RUN or Triage) — observe process tree, network, persistence.
4. **Static analysis** — file, strings, floss, hex editor - look for MZ (4D 5A), PE
5. **Network analysis** — suricata -r pcap → alerts.
6. **Check for packing** — missing strings, high entropy, section anomalies → use DIE.
7. **Unpack if needed** — upx -d or search memory dumps.
8. **Deobfuscate** — Base64 decode, .NET decompile, debug for config.
9. **Document IOCs** — hashes, domains, IPs, processes, family.

Final Mindset:

- Sandbox = fastest behavior insight.
- Dynamic analysis often bypasses obfuscation.
- Expect layers — iterate between static and dynamic.
- Goal: Extract IOCs fast for detection, hunting, and blocking.

1. Sandbox Basics & Malware Execution

When to use: First step when you have a suspicious file (doc, exe, script).
Why: Quickly see real behavior without risking your system. Combines **static** (file details) and **dynamic** (execution) analysis.

What You Upload & How It Works

- Upload the **malware/entry-point file** (e.g., malicious Word doc, .exe, .ps1).
- Sandbox runs it safely in a **virtual machine** (Windows 7/10, 32/64-bit).
- Records **everything**:
  - Processes & parent-child tree
  - Network traffic (HTTP, DNS, connections)
  - File drops & changes
  - Registry & memory activity

Key Output:

- File hashes (MD5, SHA256)
- Configuration extraction (e.g., Agent Tesla SMTP settings)
- Behavioral alerts (YARA matches)
- PCAP file (network traffic)
- Memory dumps & dropped files

Example Attack Chain (Agent Tesla):

1. Malicious Word doc opened → exploits Equation Editor (eqnedt32.exe).
2. Downloads payload via HTTP.
3. Executes via PowerShell.
4. Copies to Startup folder (persistence).
5. Checks geolocation (api.ipify.org).
6. Exfiltrates data via SMTP.

Why multiple VMs? Malware may only run on specific OS versions or require old software.

2. Safe Handling Practices

When to apply: Every time you acquire or move a malware sample. **Why**: Prevent accidental execution and spread.

Core Rules

- **ZIP archives**: Always password-protect with **infected**.
  - Prevents AV auto-deletion.
  - Clearly signals malicious content.
- **Rename extensions**: .exe → .bin or .exe.txt.
  - Stops OS auto-execution on double-click.
- **Lab setup**:
  - Dedicated host machine (never work/personal PC).
  - VMs (Windows 7/10, REMnux, Flare VM).
  - **Host-only** or air-gapped network.
  - Take VM snapshots (clean → pre-execution → during analysis).

Alternative for beginners: Use file **hash** only on VirusTotal or sandboxes instead of downloading the sample.

3. Core Static Analysis Tools (REMnux)

When to use: Immediately after receiving a sample (before running it).
Why: Verify file type, extract strings, and spot basic indicators without execution.

Tools & Commands

- **file** (Linux utility)
  Bash
  ```
  file sample.bin
  ```
  Why: Reveals true file type (e.g., RTF instead of .doc, PE for Windows executable, .NET). Ignores fake extensions.
- **strings**
  Bash
  ```
  strings sample.bin
  ```
  Why: Extracts printable ASCII strings (domains, IPs, commands). Limited to 1-byte characters.
- **floss** (better for obfuscated samples)
  Bash
  ```
  floss sample.bin
  ```
  Why: Finds Unicode (UTF-16), wide-character, and obfuscated strings.
- **Hex editor** (e.g., wxHexEditor) **Why**: Views raw bytes. Look for magic bytes:
  - MZ (4D 5A) → Windows executable start.
  - PE\0\0 (50 45 00 00) → PE header.

Key Insight: Few meaningful strings or mismatch with sandbox behavior → likely packed/obfuscated.

4. Gathering Basic Indicators (IOCs)

When to use: Early in every analysis. **Why**: Identify malware family, intent, and create detections/hunts.

Main Indicators

- **File**: Type, hashes, strings.
- **Network**: Domains, IPs, URLs, protocols.
- **Behavioral**: Processes, persistence, file drops.

Malware Stages

1. **Initial Access**: Macros, PowerShell, scripts (run once).
2. **Downloader**: Fetches next stage from internet.
3. **Dropper**: Contains payload inside (often .NET).
4. **Main Payload**: Info stealer, RAT, ransomware.

Platforms

- **VirusTotal**: Hash search, AV detections, behavior, tags.
- **MalwareBazaar** (abuse.ch): Cleaner family tags, free samples.

Goal: Quickly link to known families (e.g., Agent Tesla, RedLine, Amadey) for known capabilities.

5. Network-Based Indicators & Suricata

When to use: When you have a PCAP from sandbox or network capture.
Why: Reveals C2, downloads, exfiltration, and malware family.

Tool: Suricata

- Rule-based IDS.
- Works offline on PCAP files.

Commands

Bash
```
suricata-update                 # Download Emerging Threats rules
suricata -r sample.pcap         # Analyze PCAP (offline mode)
```

Output

- eve.json: All events (alerts, HTTP, DNS, connections). Filter:
  Bash
  ```
  grep '"alert":' eve.json
  cat eve.json | jq
  ```

Alerts examples: PE download, C2 traffic, scripting language activity.
Limitation: No matching rule = no alert.

6. Basic Dynamic Analysis

When to use: After static triage to see real behavior. **Why**: Malware must de-obfuscate and act at runtime — often bypasses static hiding.

What to Observe

- **Processes**: Parent-child chains (e.g., winword.exe → PowerShell = suspicious).
- **File system**: Drops in AppData, Temp; mass file changes (ransomware).
- **Network**: C2, exfiltration.
- **Registry/Persistence**: Scheduled tasks (schtasks.exe).

Tools:

- Sandboxes (easiest).
- Custom VMs (more control).
- Debuggers (WinDbg, x64dbg) for advanced stepping.

Anti-Analysis: Malware detects sandbox/VM and stops or acts benign.

7. Advanced Sandbox Usage (ANY.RUN)

When to use: For interactive or real-time analysis. **Why**: See live process tree, network, and threats; correlate activity easily.

Key Features

- Real-time execution view.
- Choose environment (Win7–11, 32/64-bit).
- TLS interception → see HTTP content.
- Threat tags (e.g., Amadey, Loader).

Important Tabs

- **Process**: Hierarchical tree + full command lines (e.g., schtasks.exe for persistence).
- **Network**: HTTP (downloads), Connections (IP/port), DNS.
- **Threats**: Suricata IDS alerts.

Pro options: Custom VPN/TOR, FakeNet (simulate services without real internet).

8. File Hashing & Similarity

When to use: To identify or compare samples. **Why**: Hashes are unique fingerprints for searching and sharing IOCs.

Commands (REMnux)

Bash
```
md5sum file.exe                 # Quick but weak
sha256sum file.exe              # Industry standard (MOST IMPORTANT)
sha512sum file.exe              # Stronger alternative
```

Fuzzy Hashing (ssdeep)

Bash
```
ssdeep file.exe > hash.txt
ssdeep -s -M hash.txt samples/  # Compare similarity %
```

Insight: Traditional hashes fail on packed/obfuscated variants. Fuzzy hashing finds related samples (percentage match).

9. Packing & Obfuscation Detection

When to use: When static analysis yields little (no strings, few imports).
Why: Packers hide the real payload — you analyze the wrapper instead.

Signs of Packing

- No meaningful strings (mismatch with sandbox).
- Very few imports.
- Abnormal sections (e.g., UPX0, UPX1; missing .text).
- High entropy (>7.0 — very random data).

Tools

- **Detect It Easy (DIE)**: Detects known packers (shows signatures).
- **PE Studio**: Views sections, entropy, imports.

Unpacking (UPX example)

Bash
```
upx -d file.exe                 # Unpack known UPX packer
```

Sandbox help: Search memory dumps (.dmp) for unpacked strings/domains.
Key Insight: Static ≠ dynamic behavior → almost always packed.

10. String & API Obfuscation + Cryptography

When to use: After unpacking or when strings are still hidden. **Why**: Malware hides IOCs and behavior (domains, commands, APIs).

String Obfuscation

- **Base64**: Pattern A-Za-z0-9+/= (often ends with =). Extract & decode:
  Bash
  ```
  strings --encoding=l file.exe | grep -E '[A-Za-z0-9+/=]{20,}'
  base64 -d
  ```
- Often layered with AES encryption.

API Obfuscation

- Dynamic resolution (call eax, dword_XXXX).
- Makes behavior hard to understand statically.

.NET Analysis

- Use **dnSpyEx** (decompiler + debugger on Flare VM).
- Go to entry point → find decode/decrypt functions.
- Set breakpoints → step over → read return values (e.g., C2 port, domains).

Configuration Extraction: Automate decoding to pull domains, ports, keys.

Initial Access Techniques

4/15/26 2:11 PM

Core Mindset & Attack Chain

Goal: Reconstruct the attack from initial access to impact. **Why**: Initial access malware is the entry point — simpler but heavily relies on social engineering, obfuscation, and quick-changing infrastructure.

Standard Attack Chain (Memorize This)

1. **Initial Access** — Maldoc, script, or link (phishing).
2. **Execution** — Macro (VBA) or script (PowerShell/JScript) runs.
3. **Payload Delivery** — Downloads or extracts next-stage malware (EXE/DLL).
4. **Persistence** — Scheduled task, registry, or LOLBin.
5. **C2 Communication** — Beacon, command, exfiltration (HTTP/TLS/SMTP).
6. **Impact** — Data theft, further payload, or control.

Key Insight: Early stages change rapidly and rely on user interaction. Later stages become more complex.

Safe Handling & Lab Setup

When: Every time you acquire a sample. **Why**: Prevent accidental execution or spread.

Essential Practices

- **ZIP + Password**: Always use infected. Prevents AV scanning and accidental run.
- **Rename Extension**: .exe → .exe.txt or .bin. Stops OS auto-execution.
- **Dedicated Setup**:
  - Separate analysis machine (never work/personal).
  - VMs (Flare VM recommended for Windows tools).
  - **Host-only** network (prevents lateral movement).
  - Snapshots: Clean → Pre-execution → During analysis.
- **OPSEC**: Uploading to public sandboxes (Triage, ANY.RUN) may tip off attackers. Prefer private groups when possible.

How Malware Is Delivered

Common Vectors:

- **Phishing Emails** (most common): Social engineering (fear, urgency, fake CEO/invoice).
  - Links (harder to scan) or attachments.
  - Encrypted ZIP with password in email (obfuscated: remove dashes, image, etc.).
- **Maldocs** (Word, Excel, OneNote):
  - Macros (enable content lure).
  - Template injection (loads malicious template from internet).
  - Exploits (rare, e.g., old Equation Editor).
- **Living-off-the-Land**: Uses built-in tools (PowerShell).
- **Other**: Malicious ads, SMS scams, supply-chain attacks.

Defense Layers: Email filtering, user education, EDR, least privilege, DNS/firewall blocking, network monitoring.

Threat Actors & Infrastructure

Threat Actors:

- **Nation-State (APTs)**: Sophisticated, stealthy, long campaigns.
- **Criminal Groups**: Volume-based, commodity malware (AgentTesla, RedLine, SmokeLoader).

Commodity Malware: Pre-built, sold on markets (stealers, downloaders). Check abuse.ch for top families.

Infrastructure:

- Compromised web servers (hard to block — legitimate sites).
- Abused platforms (Discord, Telegram).
- Webshells (remote shell access).
- Proxy scripts (hide real C2).
- Panels (web dashboards for managing victims).

Why it works: Fast rotation, blending with normal traffic, obfuscation.

Maldoc Analysis Workflow

Tool: oledump.py (inspects Office document structure/streams).

Steps (in order)

1. **Verify file**
   Bash
   ```
   file sample.doc
   ```
2. **Inspect structure**
   Bash
   ```
   oledump.py sample.doc
   ```
   - Look for: M (macro), O (embedded object), large streams.
3. **Extract stream**
   Bash
   ```
   oledump.py -s <index> sample.doc | more   # preview
   oledump.py -s <index> -d sample.doc > payload.bin   # dump
   ```
4. **Fix embedded PE** (if needed)
   - Open in hex editor.
   - Delete bytes before MZ (4D 5A).
   - Save as new file.
5. **Identify payload**
   Bash
   ```
   shasum -a 256 payload.bin
   ```
   Upload hash or file to Triage/ANY.RUN.

Main takeaway: Maldocs are containers. Extract the real payload or macro.

Macro & Script Analysis

Macros (VBA):

- Entry points: AutoOpen, Document_Open.
- Common patterns: CreateObject("WScript.Shell"), .exec, PowerShell download.
- Obfuscation: Random variables, string splitting (*), .replace().

Scripts (JScript/PowerShell):

- Executed by cscript.exe / wscript.exe.
- Look for: Base64 blobs, large string arrays, encoded commands.

When to debug:

- Heavy obfuscation (no visible URL/command).
- NOT needed if sandbox or static strings already show behavior.

Debugging (Macros – VBA):

- Open in Word → Enable macros.
- Developer → Visual Basic.
- Set breakpoint on key lines (CreateObject, execution).
- Run → inspect variables (hover or Watch window).

Debugging (JScript):
cmd
```
cscript //X script.js
```

- Triggers Visual Studio JIT debugger.
- Set breakpoints, continue, watch Locals window for deobfuscated strings/commands.

Main takeaway: Focus on extracting the final command/payload, not full reverse engineering.

Obfuscation & Debugging

Common Techniques:

- Base64 / hex encoding.
- Random variable/function names.
- Junk code / false flags.
- String padding + replace before execution.

When to debug: Only when static analysis fails to reveal command, URL, or payload. **Goal of debugging**: Reveal real values in memory (e.g., final PowerShell command or DLL path).

Network Analysis & Detection

What you observe:

- PowerShell/EXE downloads.
- Direct IP requests (no DNS).
- IP lookup (api.ipify.org) — geo/anti-analysis.
- POST beacons, encoded data.
- Exfiltration (passwords.txt, systeminfo).

Tools:

- **Wireshark**: Follow streams, inspect HTTP/TLS handshake.
- **Suricata**: Offline PCAP analysis, alerts on PE downloads, PowerShell, TLS certs.
  - JA3 / JA3s fingerprints (client/server behavior).
  - TLS certificate details (subject, issuer).
- **EveBox**: View Suricata alerts.

Threat Hunting Mindset:

- Single alert = weak.
- Chain = strong (download → DLLs → exfil).
- Correlate host (processes) + network (traffic).

Main takeaway: Even TLS leaves detectable patterns (certs, JA3, behavior sequences).

Overall Triage Workflow

1. **Safe handling** — ZIP + infected, rename, VM (host-only).
2. **Initial inspection** — file, oledump.py.
3. **Extract** — Macro or embedded payload.
4. **Analyze execution** — Look for AutoOpen, CreateObject, PowerShell.
5. **Debug if needed** — Reveal hidden commands (VBA or cscript //X).
6. **Identify payload** — Hash, sandbox (Triage/ANY.RUN).
7. **Network analysis** — Wireshark + Suricata for C2, downloads, exfil.
8. **Correlate** — Process chain + traffic patterns.
9. **Document IOCs** — URLs, IPs, hashes, commands, family.

Final Mindset:

- Follow the chain: Delivery → Execution → Payload → C2 → Impact.
- Combine static + dynamic + network.
- Debugging is a last resort when obfuscation hides the command.
- Goal = extract IOCs and understand behavior quickly for detection/hunting.